

En esta guía vamos a preparar nuestro entorno de trabajo para el módulo de **Desarrollo Web en Entorno Servidor (DWES)**.

Durante el curso utilizaremos **Visual Studio Code** para escribir nuestro código y **Docker** para ejecutar el servidor web y PHP.

Objetivo: al terminar esta guía podrás crear y ejecutar tu primer programa PHP desde Visual Studio Code utilizando Docker.

1. ¿Qué herramientas vamos a utilizar?

Nuestro entorno de trabajo estará formado por:

Herramienta	Para qué la utilizaremos
Visual Studio Code	Escribir y organizar nuestro código
Docker	Crear y ejecutar nuestro entorno de desarrollo
Apache	Servir nuestras páginas web
PHP	Ejecutar nuestro código PHP
Navegador web	Comprobar el resultado

La idea fundamental es:

Visual Studio Code



escribimos PHP



Docker



Apache + PHP



Navegador

Importante!!!

No vamos a instalar Apache ni PHP directamente en Windows.

Apache y PHP estarán dentro de un contenedor Docker.

2. Instalar Visual Studio Code

Visual Studio Code será nuestro editor de código durante el módulo.

Descarga e instala [Visual Studio Code](#).

Durante la instalación puedes mantener las opciones que aparecen por defecto.

Una vez finalizada la instalación, abre Visual Studio Code.

Comprobación

Debes poder acceder a la ventana principal de Visual Studio Code.

3. Configurar Visual Studio Code en español

Vamos a utilizar Visual Studio Code en **español**.

1. Abre Visual Studio Code.
2. Pulsa: **Ctrl + Shift + X**

Se abrirá el apartado de **Extensiones**.

3. Busca: **Spanish Language Pack**
4. Instala **Spanish Language Pack for Visual Studio Code**.
5. Reinicia Visual Studio Code si te lo solicita.

4. Elegir un tema

Puedes abrir el selector de temas mediante:

Ctrl + K + T

Selecciona uno de los temas disponibles.

También puedes acceder desde:

Archivo → Preferencias → Tema → Tema de color

5. Crear la carpeta de nuestro proyecto

Vamos a crear una carpeta donde guardaremos nuestro **primer proyecto**.

Por ejemplo:

C:\dwes\hola-mundo

La carpeta puede estar en otra ubicación si lo prefieres.

Durante el curso cada práctica/unidad tendrá su propia carpeta de proyecto.

6. Abrir el proyecto en Visual Studio Code

En Visual Studio Code selecciona:

Archivo → Abrir carpeta

y selecciona la carpeta:

hola-mundo

También puedes utilizar:

Ctrl + K + O

En la parte izquierda aparecerá el **Explorador**, desde donde podremos crear y organizar los archivos del proyecto.

7. Crear la carpeta src

Dentro de nuestro proyecto vamos a crear una carpeta llamada:

src

La estructura será:

hola-mundo

└─ src

En esta carpeta guardaremos nuestro código PHP.

La carpeta src será la carpeta que utilizaremos para nuestro código de aplicación.

8. Crear nuestro primer archivo PHP

Dentro de **src**, crea un archivo llamado:

index.php

La estructura será:

hola-mundo

└─ src

└─ index.php

Escribe en index.php:

<?php

echo "Hola mundo desde PHP y Docker";

?>

9. Crear el archivo compose.yaml

Ahora vamos a indicar a **Docker** cómo debe crear nuestro entorno.

En la carpeta principal del proyecto, al **mismo nivel que src**, crea:

compose.yaml

La estructura será:

hola-mundo

├─ compose.yaml

└─ src

└─ index.php

Escribe en compose.yaml:

services:

apache_php:

image: php:8.4-apache

volumes:

- ./src:/var/www/html

ports:

- "8080:80"

Este archivo será utilizado por Docker para crear nuestro servidor.

10. Entender la configuración de Docker

No necesitamos aprender Docker en profundidad todavía, pero sí debemos entender qué estamos configurando.

10.1. Apache y PHP

La siguiente línea:

image: php:8.4-apache

indica que utilizaremos una imagen que contiene:

- PHP 8.4
- Apache

Por tanto, **no necesitamos instalar PHP ni Apache en Windows.**

10.2. Nuestra carpeta src

Tenemos:

volumes:

- **./src:/var/www/html**

Esto conecta nuestra carpeta:

src

con la carpeta que utiliza Apache dentro del contenedor:

/var/www/html

Podemos representarlo así:

Nuestro ordenador Contenedor

src/index.php —————→ **/var/www/html/index.php**

Por eso, cuando Apache busque nuestros archivos, encontrará el código que estamos escribiendo en src.

10.3. El puerto 8080

Tenemos:

ports:

- **"8080:80"**

Esto significa:

Windows Contenedor

puerto 8080 —————→ **puerto 80** Apache

Por eso accederemos a nuestra aplicación mediante:

<http://localhost:8080>

11. Instalar las extensiones de Visual Studio Code

Vamos a instalar algunas extensiones que nos ayudarán durante el desarrollo.

Pulsa:

Ctrl + Shift + X

para abrir el apartado de extensiones.

11.1. PHP Intelephense

Busca:

PHP Intelephense

e instala la extensión.

Nos ayudará con:

- autocompletado.
- información sobre funciones.
- navegación por el código.
- detección de algunos errores.
- ayuda al escribir PHP.

11.2. Docker

Busca:

Docker

e instala la extensión de Docker.

Esta extensión permite trabajar con Docker desde Visual Studio Code y consultar, entre otras cosas:

- contenedores.
- imágenes.
- volúmenes.
- redes.

Importante: la extensión Docker facilita trabajar con Docker desde VS Code, pero **no sustituye a Docker Desktop** ni instala Docker por sí sola.

12. Crear la configuración de VS Code

Vamos a crear una pequeña configuración específica para nuestro proyecto.

Esto nos permitirá establecer algunos ajustes básicos del editor y trabajar todos con una configuración similar.

Dentro del proyecto crearemos una **carpeta** llamada:

.vscode

Dentro de ella crearemos un **fichero** llamado:

settings.json

La estructura completa será:

hola-mundo

├─ compose.yaml

├─ .vscode

│ └─ settings.json

└─ src

└─ index.php

En settings.json escribiremos:

```
{  
  "editor.wordWrap": "on",  
  "editor.tabSize": 4,  
  "editor.insertSpaces": true  
}
```

¿Qué significa cada opción?

editor.wordWrap

"editor.wordWrap": "on"

Hace que las líneas demasiado largas se adapten al ancho de la ventana del editor.

editor.tabSize

"editor.tabSize": 4

Establece en 4 espacios el tamaño de una tabulación.

editor.insertSpaces

"editor.insertSpaces": true

Hace que, al utilizar la tecla Tab, VS Code inserte espacios en lugar de caracteres de tabulación.

¿Por qué guardamos esta configuración dentro del proyecto?

Porque la carpeta .vscode contiene **configuraciones específicas de ese proyecto**.

De esta forma podemos tener:

Proyecto

├─ Código PHP

├─ Configuración Docker

└─ Configuración de VS Code

Esta configuración no es necesaria para que PHP o Docker funcionen. Sirve para trabajar de una forma más cómoda y uniforme.

13. Utilizar la terminal integrada de VS Code

Una de las herramientas más importantes de Visual Studio Code será la **terminal integrada**.

Podemos abrirla desde:

Terminal → Nueva terminal

o utilizando:

Ctrl + ñ

La terminal aparecerá en la parte inferior de Visual Studio Code.

Esto nos permitirá ejecutar comandos de Docker sin salir del programa.

14. Iniciar nuestro servidor

Comprueba que la terminal está situada en la carpeta del proyecto:

hola-mundo **"dir"** **"cd C:\dwes\hola-mundo"**

Ejecuta:

docker compose up -d

Docker descargará la imagen necesaria si todavía no está disponible y creará el contenedor.

La opción:

-d

hace que el contenedor se ejecute en **segundo plano**.

15. Comprobar que el contenedor está funcionando

Ejecuta:

docker ps

Debería aparecer nuestro contenedor.

No es necesario memorizar toda la información que aparece.

Lo importante es comprobar que el contenedor está funcionando.

16. Comprobar nuestro primer programa PHP

Abre un navegador web.

Escribe:

http://localhost:8080

Si todo funciona correctamente, aparecerá:

Hola mundo desde PHP y Docker

🇪🇸 ¡Nuestro entorno PHP está funcionando!

17. ¿Qué está ocurriendo realmente?

Cuando escribimos nuestro código:

```
<?php
```

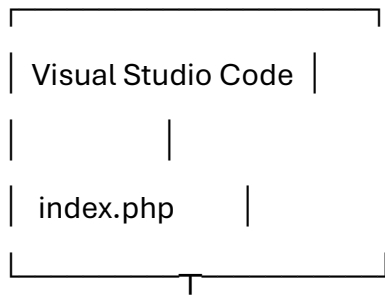
```
echo "Hola mundo desde PHP y Docker";
```

```
? >
```

Visual Studio Code **no ejecuta PHP**.

Visual Studio Code solamente nos permite escribir y editar el archivo.

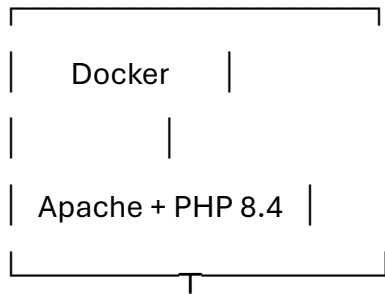
El funcionamiento es:



↓

carpeta src

↓



↓

Navegador web

↓

<http://localhost:8080>**Idea fundamental****VS Code = escribir código****Docker = proporcionar el entorno****Apache = servidor web****PHP = ejecutar nuestro código****Navegador = mostrar el resultado**

18. Detener el servidor

Cuando terminemos de trabajar podemos **detener** el **entorno**.

Ejecutamos:

docker compose down

El contenedor se detendrá.

Cuando volvamos a trabajar podremos **iniciar** nuevamente el **entorno** con:

docker compose up -d

19. Acceder al interior del contenedor

En algunas actividades necesitaremos consultar algún archivo que se encuentra dentro del contenedor.

Para ello podemos **entrar** temporalmente en él.

Desde la terminal de VS Code ejecutamos:

docker compose exec apache_php bash

Si todo funciona correctamente veremos algo parecido a:

root@a1b2c3d4:/var/www/html#

Esto significa que ahora estamos trabajando **dentro del contenedor**.

Por ejemplo, podemos consultar información de PHP con:

php --ini

También podemos consultar los archivos de configuración:

ls /usr/local/etc/php/

Y localizar el archivo de configuración

/usr/local/etc/php/php.ini-production

Y para consultar directamente algunas propiedades

```
grep -E "(file_uploads|max_execution_time|short_open_tag)" /usr/local/etc/php/php.ini-production
```

Busca líneas que comiencen por **file_uploads**, **max_execution_time** o **short_open_tag**

file_uploads → Indica si PHP permite **subir archivos al servidor mediante formularios**.

max_execution_time → Indica el **tiempo máximo, en segundos, que puede ejecutarse un script PHP**

short_open_tag → Indica si PHP permite utilizar la forma corta de abrir código PHP:

- **On** → permite `<? echo "Hola"; ?>`
- **Off** → hay que utilizar la forma recomendada `<?php echo "Hola"; ?>`

Para salir del contenedor utilizaremos:

exit

No necesitamos utilizar esta opción habitualmente. La utilizaremos únicamente en aquellas actividades en las que necesitemos consultar o realizar alguna tarea dentro del contenedor.

20. Errores y depuración básica

Durante el curso cometeremos **errores**. Es algo normal al programar.

Por ejemplo, si escribimos:

```
<?php
$nombre = "Joaquín";
echo $nombr;
?>
```

PHP mostrará un error porque hemos escrito \$nombr en lugar de \$nombre.

Visual Studio Code también nos ayuda mediante:

- colores y resaltado del código.
- autocompletado.
- detección de algunos errores.
- información sobre funciones.
- ayuda mientras escribimos.

¿Y la depuración paso a paso?

Más adelante podremos utilizar herramientas como **Xdebug** para detener la ejecución del programa y comprobar el valor de las variables paso a paso.

No necesitamos configurarlo todavía.

Lo introduciremos cuando nuestros programas sean suficientemente complejos para que tenga sentido utilizarlo.

21. Estructura final del proyecto

Al terminar toda la configuración tendremos:

hola-mundo

├─ compose.yaml

├─ .vscode

| └─ settings.json

└─ src

└─ index.php

Función de cada elemento

Elemento	Función
compose.yaml	Define nuestro entorno Docker
.vscode/	Configuración de VS Code para el proyecto
settings.json	Ajustes básicos del editor
src/	Contiene nuestro código PHP
index.php	Nuestro primer programa PHP

22. Comandos básicos

Durante el módulo utilizaremos principalmente estos comandos:

Iniciar el entorno

docker compose up -d

Comprobar los contenedores

docker ps

Detener el entorno

docker compose down

Entrar en el contenedor

docker compose exec apache_php bash

Salir del contenedor

exit

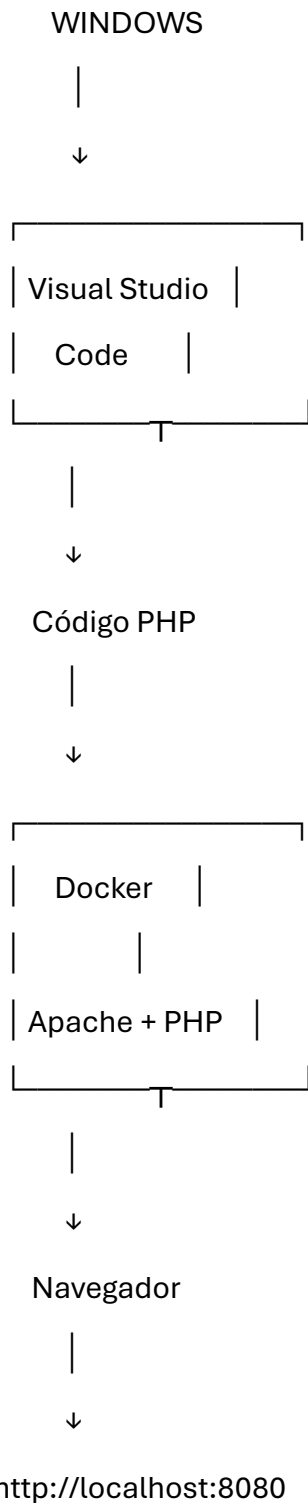
23. Comprobación final

Antes de comenzar a programar, comprueba que todos estos puntos funcionan:

- Visual Studio Code está instalado.
- Visual Studio Code está configurado en español.
- Tenemos seleccionado un tema.
- Podemos abrir la carpeta del proyecto.
- Hemos creado la carpeta src.
- Hemos creado index.php.
- Hemos creado compose.yaml.
- Hemos instalado PHP Intelephense.
- Hemos instalado la extensión Docker.
- Hemos creado .vscode/settings.json.
- Podemos abrir la terminal integrada.
- docker compose up -d funciona correctamente.
- docker ps muestra el contenedor.
- http://localhost:8080 muestra nuestro mensaje.
- Podemos detener el entorno con docker compose down.

24. Nuestro entorno de trabajo

A partir de ahora, nuestro entorno habitual será:



Este será el entorno que utilizaremos para comenzar a trabajar con **PHP en Desarrollo Web en Entorno Servidor**.

Lo que aprenderemos después

Una vez preparado el entorno, comenzaremos a trabajar con PHP:

Entorno de desarrollo



PHP básico



Variables y tipos de datos



Operadores



Condicionales



Bucles



Funciones



Formularios



Sesiones y cookies



Bases de datos



AJAX / Fetch / JSON



API REST



Laravel

Las nuevas herramientas se irán incorporando **cuando sean necesarias**, de forma progresiva.